The CalcEngine Formula Language — Formal Specification

CSC322 Group E · Design Portfolio, Part 1 of 3 Companion documents: `adt-specifications.md`, `design-portfolio.md`

The normative machine-readable grammar is `src/CalcEngine.Core/Grammar/Formula.g4`. This document is the human-readable specification of the same language: the EBNF, the lexical conventions, the precedence rationale, and — importantly for the oral defence — the places where I deliberately diverge from Excel and why.

---

1. Scope of the language

A **cell's content** is a string supplied by the client. The engine classifies it before any parsing happens:

Content	Classified as	Example
begins with `=`	*formula* — parsed with the grammar below	`=SUM(B2:B45)*0.3`
parses as a number under the invariant culture	*number literal*	`72.5`, `-3`, `1.2e3`
`TRUE` / `FALSE`, any case	*boolean literal*	`true`
one of the seven error spellings	*error literal*	`#N/A`
empty or all whitespace	*blank*	``
anything else	*text literal*	`Ngozi Okafor`

Only the first row reaches the parser. This split is deliberate: a student's name is not a syntax error, and typing `#REF!` into a cell should reproduce the error value rather than fail to parse.

---

2. EBNF

Non-terminals are `lower-camel`, terminals are `UPPER_SNAKE` or quoted literals. `?` is optional, `*` is zero-or-more, `|` is alternation.

```
formula        ::= '=' expression EOF ;

expression     ::= unary ;

(* precedence layers, loosest first - see §4 for the encoding actually used *)
unary          ::= ( '+' | '-' ) expression
                 | percent ;
percent        ::= power '%'* ;
power          ::= multiplicative ( '^' power )? ;          (* right associative *)
multiplicative ::= additive ( ( '*' | '/' ) additive )* ;
additive       ::= concatenation ( ( '+' | '-' ) concatenation )* ;
concatenation  ::= comparison ( '&' comparison )* ;
comparison     ::= atom ( ( '=' | '<>' | '<' | '<=' | '>' | '>=' ) atom )* ;

atom           ::= NUMBER
                 | STRING
                 | TRUE | FALSE
                 | ERROR_LITERAL
                 | functionCall
                 | reference
                 | '(' expression ')' ;

functionCall   ::= functionName '(' argumentList? ')' ;
functionName   ::= IDENTIFIER | CELL_REF ;
argumentList   ::= expression ( ',' expression )* ;

reference      ::= SHEET_QUALIFIER? CELL_REF ':' CELL_REF   (* range  *)
                 | SHEET_QUALIFIER? CELL_REF ;              (* single *)
```

The layered presentation above is the classical way of writing precedence into an EBNF. ANTLR 4 expresses the same language more compactly with a single left-recursive rule whose alternatives are ordered by precedence; that is what `Formula.g4` contains, and §4 explains the correspondence.

---

3. Lexical conventions

Terminal	Definition	Notes
`NUMBER`	`( [0-9]+ ( '.' [0-9]* )? | '.' [0-9]+ ) ( [Ee] [+-]? [0-9]+ )?`	Invariant culture only; `1,5` is *not* a number. No sign — `-3` is unary minus applied to `3`.
`STRING`	`'"' ( ~'"' | '""' )* '"'`	A literal quote is doubled: `"she said ""yes"""`.

Terminal	Definition	Notes
`TRUE` / `FALSE`	case-insensitive keywords	`True`, `TRUE`, `true` all lex as the boolean.
`ERROR_LITERAL`	`#DIV/0!` `#VALUE!` `#REF!` `#NAME?` `#NUM!` `#N/A` `#CIRC!`	Upper case only, as spelled by the engine itself.
`CELL_REF`	`'$'? [A-Za-z]+ '$'? [0-9]+`	Column letters are case-insensitive: `b2` ≡ `B2`. `$` marks an absolute component.
`SHEET_QUALIFIER`	`( [A-Za-z_][A-Za-z0-9_.]* | '\'' ( ~'\'' | '\'\'' )* '\'' ) '!'`	Carries its own `!`; see §5.
`IDENTIFIER`	`[A-Za-z_][A-Za-z0-9_.]*`	Function names.
`WS`	`[ \t\r\n]+`	Skipped everywhere; `= SUM ( B2 : B45 )` is legal.
`UNTERMINATED_STRING`	`'"' ( ~'"' | '""' )*`	Exists only to produce a good error message.
`UNEXPECTED_CHAR`	`.`	Catch-all, see §6.

Bounds. A `CELL_REF` is *lexically* any letters-then-digits. The bounds check (`A`–`XFD`, rows `1`–`1048576`) is done in the AST builder, not the lexer, so that `ZZZZ9` produces the message *"column 'ZZZZ' is beyond the last column XFD"* rather than a bare "no viable alternative".

---

4. Precedence and associativity

Highest binding first. This is **Excel's** table, not C's.

Level	Operators	Associativity
1	unary `+` `−`	prefix
2	`%`	postfix
3	`^`	**right**
4	`*` `/`	left
5	`+` `−`	left
6	`&`	left
7	`=` `<>` `<` `<=` `>` `>=`	left

In ANTLR 4 a left-recursive rule resolves ambiguity by *alternative order*: the alternative listed first wins, i.e. binds tightest. `Formula.g4` therefore lists `UnaryExpression` first and `ComparisonExpression` last, which encodes exactly the table above. `<assoc=right>` is applied to `^` only.

4.1 The `-2^2` decision

`=-2^2` evaluates to **4** in CalcEngine, because unary minus binds tighter than `^` — the same as Excel, LibreOffice and Google Sheets, and the opposite of the convention in mathematics and in C.

I chose Excel compatibility over mathematical convention because the engine's declared purpose is to run formulas copied out of existing departmental workbooks. A workbook that silently changes the sign of a term when it is migrated is worse than one that is surprising to a mathematician. A one-line change (moving the `UnaryExpression` alternative below `PowerExpression`) flips the decision, and `PrecedenceTests.Negation_BindsTighterThanPower_LikeExcel` documents it as a test rather than as folklore.

4.2 Comparisons are non-chaining in practice

`=1<2<3` parses (left-associatively) as `(1<2)<3` → `TRUE<3` → `FALSE`, because a boolean sorts above any number in the spreadsheet ordering. This is Excel's behaviour and falls out of the grammar; I did not special-case it.

---

5. Two lexical decisions worth defending

Sheet qualifiers carry their `!`. `Marks!B2` would otherwise be ambiguous: `Marks` matches `CELL_REF` (column `MARKS`, row… no digits — so `IDENTIFIER`), and the parser would need a token of lookahead in the lexer, which ANTLR's lexer does not do. By making `Marks!` a single 6-character token, maximal munch decides it for us with no ambiguity and no semantic predicates.

Function names may be lexed as `CELL_REF`. `LOG10(` lexes as `CELL_REF LPAREN`, not `IDENTIFIER LPAREN`, because `LOG10` matches the cell reference pattern and both rules match the same length (declaration order then picks `CELL_REF`). Rather than fight the lexer with predicates, the parser rule `functionName : IDENTIFIER | CELL_REF` accepts either and the following `(` disambiguates. This is precisely how a real spreadsheet resolves `LOG10` — the `(` makes it a call — and it means the function library can grow to include digit-bearing names without touching the grammar.

Deliberate exclusions. Whole-column ranges (`A:A`), 3-D ranges (`Sheet1:Sheet3!A1`), defined names, array formulas and structured table references are *not* in the language. `A:A` in particular was rejected on design grounds, not effort: a whole-column reference makes a single dependency edge stand for 1,048,576 cells, which defeats the range-dependency index described in the ADT specification and would put the 50 ms propagation target out of reach. `SUM(A1:A1000)` is the supported spelling.

---

6. Error reporting

Malformed input is normal input. Three properties are required of every rejection:

1. **It never throws to the client.** `FormulaParser.Parse` returns a `ParseResult`, which is either a tree or a non-empty list of `FormulaSyntaxError`. Exceptions are reserved for engine bugs.
2. **It says where.** Every error carries a 1-based `Column` into the original cell content — the `=` included, so the column can be used directly to place a caret in the GUI's formula bar.

3. **It says what.** ANTLR's default ("mismatched input ')' expecting {...}") is translated by `DescriptiveErrorListener` into the vocabulary a spreadsheet user has: *value*, *cell reference*, *function*, *operator*, *closing bracket*.

The `UNEXPECTED_CHAR` catch-all rule exists for property 1. Without it the lexer has its own error channel, and a formula such as `=A1 § 2` produces one lexer error *and* one parser error at different positions. With it, every character becomes a token and there is exactly one error path.

Examples produced by the implementation (`SyntaxErrorMessageTests`):

Input	Message
`=SUM(B2:B45`	`Column 12: the bracket opened at column 5 was never closed.`
`=2 +`	`Column 5: a value, cell reference or function call was expected after '+'.`
`=SUM(,1)`	`Column 6: a value, cell reference or function call was expected, but ',' was found.`
`=A1 $ B2`	`Column 5: '$' is not a valid character in a formula.`
`="unterminated`	`Column 2: this text literal has no closing quotation mark.`
`=ZZZZ9+1`	`Column 2: 'ZZZZ' is beyond the last column, XFD.`
`=NOSUCH(1)`	`Column 2: there is no function called 'NOSUCH'.` (evaluation-time `#NAME?`)

---

7. Worked derivation

`=SUM(B2:B45)*0.3`

```
formula
└── '=' expression EOF
        └── MultiplicativeExpression                    (level 4)
            ├── AtomExpression → FunctionCallAtom
            │      functionName IDENTIFIER "SUM"
            │      '(' argumentList ')'
            │           └── ReferenceAtom → RangeReference
            │                    CELL_REF "B2" ':' CELL_REF "B45"
            ├── op = '*'
            └── AtomExpression → NumberLiteral 0.3
```

which the AST builder lowers to

```
BinaryOperatorNode(Multiply)
├── FunctionCallNode("SUM")
│    └── RangeReferenceNode(B2:B45)
└── NumberLiteralNode(0.3)
```

and whose dependency set is the single range `B2:B45` — one edge in the range index, not 44 edges in the adjacency list.